# CS214 Bonus Assignment

Hugo Wan
NetID: twan012

Deadline: 5/28/2026

## 1 Introduction

In this bonus assignment, the problem is about finding the "magic number" of a hidden number. The magic number of a number $x$ means the number of positive factors of $x$. For example, if $x = 15$, then its factors are $1, 3, 5$, and $15$, so the magic number is 4.

There are two parts to solve. First, I need to design a parallel algorithm that computes the magic number for every number from 1 to $n$. The required bound is $O(n \log n)$ work and polylogarithmic span. Second, for the specific case $n = 50$, I need to give a yes-or-no question strategy to find the magic number with a low expected number of guesses.

## 2 Task 1: Parallel Algorithm for General $n$

### 2.1 Idea

A simple way to find the number of factors of one number $x$ is to test every possible divisor. However, doing that separately for all numbers from 1 to $n$ would repeat a lot of work.

Instead, I use a sieve-like method. The idea is: for each possible divisor $d$, add 1 to the factor count of all multiples of $d$.

For example, if $d = 3$, then 3 is a factor of:

$$3, 6, 9, 12, 15, \ldots$$

So we add 1 to the factor count of all of those numbers.

This way, every divisor contributes to all numbers that it divides.

### 2.2 Algorithm

Let `factor_count`$[1 \ldots n]$ be an array. At the end, `factor_count`$[x]$ should store the number of positive factors of $x$.

1. Initialize `factor_count`$[1 \ldots n]$ to all 0.
2. In parallel, loop over all possible divisors $d = 1, 2, \ldots, n$.
3. For each $d$, in parallel, visit all multiples of $d$:

$$d, 2d, 3d, \ldots, \left\lfloor \frac{n}{d} \right\rfloor d.$$

4. For each multiple $k$, add 1 to `factor_count`$[k]$.

5. Return `factor_count`.

The pseudocode is:

```
ComputeMagicNumbers(n) :
  Create factor_count[1...n] and initialize all entries to 0
  parallel for d = 1 to n :
    parallel for j = 1 to ⌊n/d⌋ :
      k = j · d
      factor_count[k] = factor_count[k] + 1
  return factor_count
```

There is one implementation detail. Different divisors might update the same `factor_count`$[k]$ at the same time. For example, 12 can be updated by $d = 1, 2, 3, 4, 6, 12$. So in an actual parallel implementation, I would use atomic add or use a parallel reduction method to safely combine the updates.

### 2.3 Correctness

I will explain why this algorithm gives the correct magic number.

Take any number $x$, where $1 \leq x \leq n$. The algorithm increases `factor_count`$[x]$ whenever the current divisor $d$ has $x$ as one of its multiples. This happens exactly when $d$ divides $x$.

So the final value of `factor_count`$[x]$ is:

$$\texttt{factor\_count}[x] = |\{d : 1 \leq d \leq x \text{ and } d \mid x\}|.$$

This set is exactly the set of positive factors of $x$. Therefore, `factor_count`$[x]$ is the number of factors of $x$, which is the magic number.

Since this argument works for every $x$ from 1 to $n$, the algorithm correctly computes all magic numbers.

### 2.4 Work Analysis

For a fixed divisor $d$, the inner loop visits all multiples of $d$ up to $n$. The number of such multiples is:

$$\left\lfloor \frac{n}{d} \right\rfloor.$$

Therefore, the total work is:

$$\sum_{d=1}^{n} \left\lfloor \frac{n}{d} \right\rfloor.$$

This can be bounded by:

$$\sum_{d=1}^{n} \left\lfloor \frac{n}{d} \right\rfloor \leq \sum_{d=1}^{n} \frac{n}{d}.$$

Then:

$$\sum_{d=1}^{n} \frac{n}{d} = n \sum_{d=1}^{n} \frac{1}{d}.$$

The summation

$$\sum_{d=1}^{n} \frac{1}{d}$$

is the harmonic series, which is $O(\log n)$.

So the total work is:

$$O(n \log n).$$

This matches the required work bound.

### 2.5 Span Analysis

The algorithm is parallel because the loop over $d$ can be run in parallel. Also, for each $d$, the multiples of $d$ can be processed in parallel.

If we use fork-join parallel loops, each parallel loop adds logarithmic span. The update step can be handled using atomic operations or a parallel reduction. With these standard parallel operations, the span is polylogarithmic.

For example, a reasonable span bound is:

$$O(\log^2 n).$$

The exact span may depend on the implementation, but it is still polylogarithmic. Therefore, the algorithm satisfies the requirement:

$$\boxed{O(n \log n) \text{ work and polylogarithmic span.}}$$

## 3 Task 2: Strategy for $n = 50$

### 3.1 Grouping the Magic Numbers

For $n = 50$, the assignment gives the number of factors for each number from 1 to 50. I first group the numbers by their magic number.

| Magic Number | Frequency |
|---|---|
| 1 | 1 |
| 2 | 15 |
| 3 | 4 |
| 4 | 15 |
| 5 | 1 |
| 6 | 8 |
| 8 | 4 |
| 9 | 1 |
| 10 | 1 |

The total is:

$$1 + 15 + 4 + 15 + 1 + 8 + 4 + 1 + 1 = 50.$$

So the possible magic numbers are:

$$\{1, 2, 3, 4, 5, 6, 8, 9, 10\}.$$

Since the original number is equally likely to be any number from 1 to 50, these frequencies also tell us how likely each magic number is. The most common magic numbers are 2 and 4, and both appear 15 times. The next most common one is 6, which appears 8 times.

Because of this, I want my decision tree to find 2, 4, and 6 quickly. The rare magic numbers can take more questions because they happen less often.

### 3.2 Question Strategy

My yes-or-no strategy is the following.

**Q1.** Is the magic number either 2 or 4?

- If yes, ask Q2.
- If no, ask Q3.

**Q2.** Is the magic number 4?

- If yes, the magic number is 4.
- If no, the magic number is 2.

**Q3.** Is the magic number either 3 or 8?

- If yes, ask Q4.
- If no, ask Q5.

**Q4.** Is the magic number 8?

- If yes, the magic number is 8.
- If no, the magic number is 3.

**Q5.** Is the magic number 6?

- If yes, the magic number is 6.
- If no, ask Q6.

**Q6.** Is the magic number either 9 or 10?

- If yes, ask Q7.
- If no, ask Q8.

**Q7.** Is the magic number 10?

- If yes, the magic number is 10.
- If no, the magic number is 9.

**Q8.** Is the magic number 5?

- If yes, the magic number is 5.
- If no, the magic number is 1.

This strategy is not trying to make every answer take the same number of questions. Instead, it tries to make common answers shorter. That is why 2 and 4 only need 2 questions, while the rare answers can need more questions.

### 3.3 Expected Number of Questions

The number of questions needed for each magic number is:

| Magic Number | Frequency | Questions Needed |
|---|---|---|
| 2 | 15 | 2 |
| 4 | 15 | 2 |
| 3 | 4 | 3 |
| 8 | 4 | 3 |
| 6 | 8 | 3 |
| 1 | 1 | 5 |
| 5 | 1 | 5 |
| 9 | 1 | 5 |
| 10 | 1 | 5 |

So the expected number of questions is:

$$E = \frac{15(2) + 15(2) + 4(3) + 4(3) + 8(3) + 1(5) + 1(5) + 1(5) + 1(5)}{50}.$$

Then:

$$E = \frac{30 + 30 + 12 + 12 + 24 + 5 + 5 + 5 + 5}{50}.$$

So:

$$E = \frac{128}{50}.$$

Therefore:

$$E = 2.56.$$

So this decision strategy needs:

$$\boxed{2.56}$$

questions on average.

### 3.4 Why This Strategy Makes Sense

This strategy is based on the same idea as a Huffman-style decision tree. Since each yes-or-no question creates a binary decision tree, the expected number of questions is like the weighted external path length of the tree. To minimize the average number of questions, the more frequent magic numbers should be placed closer to the root, while the rare magic numbers can be placed deeper.

The frequencies are:

$$15, 15, 8, 4, 4, 1, 1, 1, 1.$$

A Huffman-style grouping gives an average cost of:

$$\frac{128}{50} = 2.56.$$

My decision strategy has this same weighted average cost, so it achieves this low expected number of questions.

In this strategy:

- 2 and 4 appear most often, so they only need 2 questions.
- 6 appears next most often, so it only needs 3 questions.
- The rare values appear only once, so it is acceptable if they need 5 questions.

This keeps the average number of questions low.

# 4 Conclusion

In this assignment, I first designed a parallel sieve-style algorithm to compute the number of factors for every number from 1 to $n$. The idea is to let each possible divisor $d$ contribute 1 to all multiples of $d$. This gives the correct magic number for every $x$, because each number receives one count from each of its positive divisors.

The total work is:

$$O(n \log n),$$

and the span is polylogarithmic using parallel loops and safe parallel updates.

For $n = 50$, I grouped the possible hidden numbers by their magic number and built a yes-or-no question strategy. The strategy asks about common magic numbers earlier and rare magic numbers later. The expected number of questions is:

$$\boxed{2.56}.$$

# References


1. Yihan Sun, *CS214 Bonus Assignment Handout: The Magic Number*, CS214 Spring 2026.
2. Yihan Sun, *Our First Parallel Algorithm: Algorithm, Analysis, Cost Model, and Implementation*, CS214 Parallel Algorithms Lecture Slides.
3. Yihan Sun, *Implementing Parallel Algorithms*, CS214 Parallel Algorithms Lecture Slides.
4. Yihan Sun, *Race and Concurrency*, CS214 Parallel Algorithms Lecture Slides.
5. OpenAI. ChatGPT (GPT-5.5 Thinking). Used for helping draft explanations, checking clarity, and formatting the report in LaTeX.
   `https://chatgpt.com/s/t_6a18c14177d8819190555d6d95bf8792`